

COMP3023: Design and Analysis of Algorithms

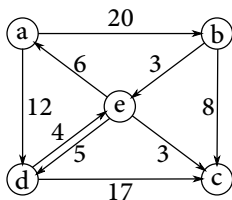
Lecture 12: All-Pairs Shortest Path

Computer Science and Technology
United International College

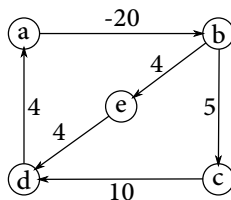
- 1 Problem description
- 2 Trivial Solution
- 3 Dynamic Programming
- 4 Improvemet
- 5 The Floyd-Warshall Algorithm

Problem description

Given a weighted digraph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, ($\mathbb{R}$ is the set of real numbers), determine the length of the shortest path (distance) between all pairs of vertices in G . Here we assume that there are no cycles with zero or negative cost.



(a) Without negative cost cycle



(b) With negative cost cycle

Using Dijkstra's Algorithm

- Apply Dijkstra's algorithm as a subroutine on every vertex of the digraph.
- Dijkstra's algorithm from a single source runs in $\Theta((n + e) \log n)$.
- For all vertices, $\Theta(n(n + e) \log n) = \Theta(n^2 \log n + ne \log n)$.
- If the digraph is dense (many edges), this is an $\Theta(n^3 \log n)$ algorithm.
- With advanced (but complicated) data structures, D's algorithm runs in $\Theta(n \log n + e)$ time yielding a $\Theta(n^2 \log n + ne)$ algorithm. For dense graphs this is $\Theta(n^3)$ time.

Solution 2: Dynamic Programming

- 1 How do we decompose the all-pairs shortest paths problem into subproblems?
- 2 How do we express the optimal solution of a subproblem in terms of optimal solutions to some subsubproblems?
- 3 How do we use the recursive relation from (2) to compute the optimal solution in a bottom-up fashion?
- 4 How do we construct all the shortest paths?

Solution 2: Input and Output Formats

To simplify the notation, we assume that $V = \{1, 2, \dots, n\}$.

Assume that the graph is represented by an $n \times n$ matrix with the weights of the edges:

$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ w(i, j) & \text{if } i \neq j \text{ and } (i, j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E. \end{cases}$$

Output Format: an $n \times n$ matrix $D = [d_{ij}]$ where d_{ij} is the length of the shortest path from vertex i to j .

Step 1: Decompose the original problem

Decomposition guidelines:

- Subproblems with smaller sizes should be easier to solve.
- An optimal solution to a subproblem should be expressed in terms of the optimal solutions to subproblems with smaller sizes.

Step 1: Decomposition in a **Natural** way

- Define $d_{ij}^{(m)}$ to be the length of the **shortest path** from i to j that **contains at most m edges**.
- Let D^m be the $n \times n$ matrix $[d_{ij}^{(m)}]$.
- $d_{ij}^{(n-1)}$ is the **true distance** from i to j (see the next page for a proof of this conclusion).
- **Subproblems:** compute $D^{(m)}$ for $m = 1, 2, \dots, n - 1$.

Question: Which $D^{(m)}$ is easiest to compute?

Lemma

$d_{ij}^{(n-1)}$ defined above is the true distance from i to j .

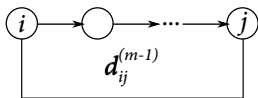
Proof: We prove that any shortest path P from i to j contains at most $n - 1$ edges.

First note that since all cycles have positive weight, a shortest path can have no cycles (if there were a cycle, we could remove it and lower the length of the path).

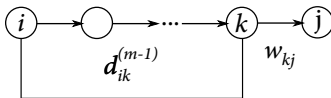
A path without cycles can have length at most $n - 1$ (since a longer path must contain some vertex twice, that is, contain a cycle).

Step 2: Recurrence relation

Consider a **shortest path** from i to j of length $d_{ij}^{(m)}$.



(a) **Case 1:** at most $m - 1$ edges shortest path



(b) **Case 2:** exactly m edges

Case 1: It has at most $m - 1$ edges. Then

$$d_{ij}^{(m)} = d_{ij}^{(m-1)} = d_{ij}^{(m-1)} + w_{jj}$$

Case 2: It has m edges. Let k be the vertex before j on a shortest path. Then

$$d_{ij}^{(m)} = d_{ik}^{(m-1)} + w_{kj}$$

Combining the two cases

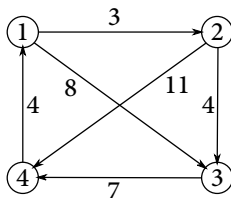
$$d_{ij}^{(m)} = \min_{1 \leq k \leq n} \{d_{ik}^{(m-1)} + w_{kj}\}$$

Step 3: Bottom-up Computation of $D^{(n-1)}$

- Bottom: $D^{(1)} = [w_{ij}]$, the weight matrix.
- Compute $D^{(m)}$ from $D^{(m-1)}$, for $m = 2, \dots, n-1$, using

$$d_{ij}^{(m)} = \min_{1 \leq k \leq n} \{d_{ik}^{(m-1)} + w_{kj}\}$$

Example: Bottom-up Computation of $D^{(n-1)}$

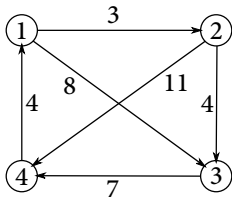


$D^{(1)} = [w_{ij}]$ is just the weight matrix:

$$D^{(1)} = \begin{bmatrix} 0 & 3 & 8 & \infty \\ \infty & 0 & 4 & 11 \\ \infty & \infty & 0 & 7 \\ 4 & \infty & \infty & 0 \end{bmatrix}$$

Example: Computing $D^{(2)}$ from $D^{(1)}$

$$d_{ij}^{(2)} = \min_{1 \leq k \leq 4} \{d_{ik}^{(1)} + w_{kj}\}$$

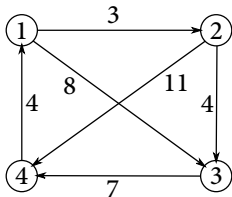


With $D^{(1)}$ given earlier and the recurrence relation,

$$D^{(2)} = \begin{bmatrix} 0 & 3 & 7 & 14 \\ 15 & 0 & 4 & 11 \\ 11 & \infty & 0 & 7 \\ 4 & 7 & 12 & 0 \end{bmatrix}$$

Example: Computing $D^{(3)}$ from $D^{(2)}$

$$d_{ij}^{(3)} = \min_{1 \leq k \leq 4} \{d_{ik}^{(2)} + w_{kj}\}$$



With $D^{(2)}$ given earlier and the recurrence relation,

$$D^{(3)} = \begin{bmatrix} 0 & 3 & 7 & 14 \\ 15 & 0 & 4 & 11 \\ 11 & 14 & 0 & 7 \\ 4 & 7 & 11 & 0 \end{bmatrix}$$

$D^{(3)}$ gives the distances between any pair of vertices.

The Algorithm for Computing $D^{(n-1)}$

Algorithm 1: Computing $D^{(n-1)}$

Input: Digraph $G = (V, E)$ and the weight matrix $D^{(1)} = [w_{ij}]$.

Output: All pairs shortest path $D^{(n-1)}$.

```
1 begin
2   for  $m = 2$  to  $n - 1$  do
3     for  $i = 1$  to  $n$  do
4       for  $j = 1$  to  $n$  do
5         min  $\leftarrow \infty$ ;
6         for  $k = 1$  to  $n$  do
7           new  $\leftarrow d_{ik}^{(m-1)} + w_{kj}$ ;
8           if new  $<$  min then
9             min  $\leftarrow$  new;
10         $d_{ij}^{(m)} \leftarrow$  min;
```

Comments on the Current Dynamic Programming

- Algorithm uses $\Theta(n^3)$ space; how can this be reduced down to $\Theta(n^2)$?
- How can we extract the actual shortest paths from the solution?
- Running time $O(n^4)$, much worse than the solution using Dijkstra's algorithm. Can we improve this?

Repeated Squaring

Observe that we are only interested to find $D^{(n-1)}$, all others D^i , $1 \leq i \leq n-2$ are only auxiliary. Furthermore, since the graph does not have negative cycle, we have $D^{(n-1)} = D^i$, for all $i \geq n$.

In particular, this implies that $D^{(2^{\lceil \log_2 n \rceil})} = D^{(n-1)}$.

We can calculate $D^{(2^{\lceil \log_2 n \rceil})}$ using “repeated squaring” to find

$$D^{(2)}, D^{(4)}, \dots, D^{(2^{\lceil \log_2 n \rceil})}$$

Repeated Squaring

We use the recurrence relation:

- Bottom: $D^{(1)} = [w_{ij}]$, the weight matrix.
- For $s \geq 1$ compute $D^{(2^s)}$ using

$$d_{ij}^{(2^s)} = \min_{1 \leq k \leq n} \{d_{ik}^{(s)} + d_{kj}^{(s)}\}$$

Given this relation we can calculate $D^{(2^i)}$ from $D^{(2^{i-1})}$ in $O(n^3)$ time. We can therefore calculate **all** of

$$D^{(2)}, D^{(4)}, \dots, D^{(2^{\lceil \log_2 n \rceil})}$$

in $O(n^3 \log n)$ time, improving our running time.

The Floyd-Warshall Algorithm

Step 1: Decomposition

Definition

The vertices $v_2, v_3, \dots, v_{l-1}$ are called the *intermediate vertices* of the path $p = \langle v_1, v_2, \dots, v_{l-1}, v_l \rangle$.

- Let $d_{ij}^{(k)}$ be the **length of the shortest path** from i to j such that *all* intermediate vertices on the path (if any) are in set $\{1, 2, \dots, k\}$.
 $d_{ij}^{(0)}$ is set to be w_{ij} , i.e., no intermediate vertex. Let $D^{(k)}$ be the $n \times n$ matrix $[d_{ij}^{(k)}]$.
- Claim: $d_{ij}^{(n)}$ is the distance from i to j . So our aim is to compute $D^{(n)}$.
- Subproblems:** compute $D^{(k)}$ for $k = 0, 1, \dots, n$.

Step 2: Structure of shortest paths

Observation

A shortest path does not contain the same vertex twice.

Proof sketch: A path containing the same vertex twice contains a cycle. Removing cycle gives a shorter path.

Observation

For a shortest path from i to j such that any intermediate vertices on the path are chosen from the set $\{1, 2, \dots, k\}$, there are two possibilities:

- *If k is not a vertex on the path, the shortest such path has length $d_{ij}^{(k-1)}$.*
- *If k is a vertex on the path, the shortest such path has length $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.*

Step 2: Structure of shortest paths

Consider a **shortest path** from i to j containing the vertex k . It consists of a subpath from i to k and a subpath from k to j . Each subpath can only contain intermediate vertices in $\{1, \dots, k-1\}$, and must be as short as possible, namely they have lengths $d_{ik}^{(k-1)}$ and $d_{kj}^{(k-1)}$.

Hence the path has length $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Combining the two cases we get

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

Step 3: the Bottom-up Computation

- Bottom: $D^{(0)} = [w_{ij}]$, the weight matrix.
- Compute $D^{(k)}$ from $D^{(k-1)}$ using

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

for $k = 1, \dots, n$.

The Floyd-Warshall Algorithm V1

Algorithm 2: The Floyd-Warshall Algorithm V1

Input: Digraph $G = (V, E)$ on n vertices and the weight matrix $[w_{ij}]$.

Output: All pairs shortest path matrix $d^{(n)}$.

```
1 begin
2   for  $i = 1$  to  $n$  do
3     for  $j = 1$  to  $n$  do
4        $d^0[i, j] \leftarrow w[i, j]$ ;
5        $pred[i, j] \leftarrow nil$ ;
6   for  $k = 1$  to  $n$  do
7     for  $i = 1$  to  $n$  do
8       for  $j = 1$  to  $n$  do
9         if  $d^{(k-1)}[i, k] + d^{(k-1)}[k, j] < d^{(k-1)}[i, j]$  then
10           $d^{(k)}[i, j] \leftarrow d^{(k-1)}[i, k] + d^{(k-1)}[k, j]$ ;
11           $pred[i, j] \leftarrow k$ ;
12        else
13           $d^{(k)}[i, j] \leftarrow d^{(k-1)}[i, j]$ 
14  return  $d^{(n)}[1 \dots n, 1 \dots n]$ ;
```

Comments on the Floyd-Warshall Algorithm V1

- The algorithm's running time is clearly $\Theta(n^3)$
- The predecessor pointer $pred[i, j]$ can be used to extract the final path (see later).
- Problem: the algorithm uses $\Theta(n^3)$. It is possible to reduce this down to $\Theta(n^2)$ space by keeping only one matrix instead of n .
- The algorithm is on the next page. Convince yourself that it works.

The Floyd-Warshall Algorithm V2

Algorithm 3: The Floyd-Warshall Algorithm V2

Input: Digraph $G = (V, E)$ on n vertices and the weight matrix $[w_{ij}]$.

Output: All pairs shortest path matrix d .

```
1 begin
2   for  $i = 1$  to  $n$  do
3     for  $j = 1$  to  $n$  do
4        $d[i, j] \leftarrow w[i, j]$ ;
5        $pred[i, j] \leftarrow nil$ ;
6   for  $k = 1$  to  $n$  do
7     for  $i = 1$  to  $n$  do
8       for  $j = 1$  to  $n$  do
9         if  $d[i, k] + d[k, j] < d[i, j]$  then
10            $d[i, j] \leftarrow d[i, k] + d[k, j]$ ;
11            $pred[i, j] \leftarrow k$ ;
12 return  $d[1 \dots n, 1 \dots n]$ ;
```

Extracting the Shortest Paths

The predecessor pointers $\text{pred}[i, j]$ can be used to extract the final path. The idea is as follows.

- Whenever we discover that the shortest path from i to j passes through an intermediate vertex k , we set $\text{pred}[i, j] = k$.
- If the shortest path does not pass through any intermediate vertex, then $\text{pred}[i, j] = \text{nil}$.
- To find the shortest path from i to j , we consult $\text{pred}[i, j]$. If it is nil , then the shortest path is just the edge (i, j) . Otherwise, we recursively compute the shortest path from i to $\text{pred}[i, j]$ and the shortest path from $\text{pred}[i, j]$ to j .

The Algorithm for Extracting the Shortest Paths

Algorithm 4: $Path(i, j)$

Input: Predecessor matrix $pred$.

Output: The shortest path from i to j .

```
1 begin
2   if  $pred[i, j] = nil$  then
3      $\_output(i, j);$ 
4   else
5      $Path(i, pred[i, j]);$ 
6      $Path(pred[i, j], j);$ 
```

Example of Extracting the Shortest Paths

Find the shortest path from vertex 2 to vertex 3.

2...3	$Path(2, 3)$	$pred[2, 3] = 4$	
2...4...3	$Path(2, 4)$	$pred[2, 4] = 5$	
2...5...4...3	$Path(2, 5)$	$pred[2, 5] = nil$	$Output(2, 5)$
25...4...3	$Path(5, 4)$	$pred[5, 4] = nil$	$Output(2, 5)$
254...3	$Path(4, 3)$	$pred[4, 3] = 6$	
254...6...3	$Path(4, 6)$	$pred[4, 6] = nil$	$Output(4, 6)$
2546...3	$Path(6, 3)$	$pred[6, 3] = nil$	$Output(6, 3)$
25463			